Engineering Secure Software Systems

Recap

Henri Heyden

stu240825

Christian-Albrechts-Universität zu Kiel

Winter Term 2025/26

Contents

1 Cryptographic Protocols **5**
1.1 Foundations 5
1.2 Formal Protocol Model 6
1.2.1 Formal Protocol Model 10
1.2.1.1 Needham-Schroeder Protocol 10
1.2.2 Modeling Protocol Attacks 10
1.3 Rusinowitch-Turuani Theorem 12
1.3.1 Using Directed Acyclic Graphs to Model Terms 12
1.3.2 Short Attacks 13
1.4 Automatic Analysis 14
1.4.1 ProVerif 16

2 Information Flow **19**
2.1 Formal Model 19
2.2 Security Models 20

Chapter 1

Cryptographic Protocols

This lecture is about the security of message protocols. There are different definitions of security, in the course of this lecture we define, what security means in this context of cryptographic protocols, and how to automatically verify the security of a protocol.

1.1 Foundations

First and foremost, we define the cryptographic primitives and basic operations.

Definition 1.1.1 (Cryptographic Primitives: Encryption)**.** For $X, Y, K \subseteq \Sigma^*$ for alphabet Σ, we define the *symmetric* encryption $\mathtt{enc}^{\mathrm{s}} : X \times K \to Y$ and decryption $\mathtt{dec}^{\mathrm{s}} : Y \times K \to X$ as some bijective functions with

$$\mathtt{dec}^{\mathrm{s}}_k \circ \mathtt{enc}^{\mathrm{s}}_k = x \mapsto x.$$

Furthermore, for $\hat{K} \subseteq \Sigma^*$ and some bijective mapping to K, we define the *asymmetric* encryption $\mathtt{enc}^{\mathrm{a}} : X \times K \to Y$ and decryption $\mathtt{dec}^{\mathrm{a}} : Y \times \hat{K} \to X$ functions with property

$$\mathtt{dec}^{\mathrm{a}}_{\hat{k}} \circ \mathtt{enc}^{\mathrm{a}}_k = x \mapsto x.$$

In both encryption primitives, we assume the security property that there is no easily computable direct mapping $Y \to X$, nor $K \to \hat{K}$.

Definition 1.1.2 (Cryptographic Primitives: Signatures)**.** For $X, K, T \subseteq \Sigma^*$, we define the symmetric signature $\mathtt{mac} : X \times K \to T$ and verification $\mathtt{test}^{\mathrm{s}} : X \times K \times T \to \{\mathtt{ok}, \mathtt{error}\}$, such that

$$\mathtt{test}^{\mathrm{s}}_k(x, \mathtt{mac}_k(x)) \overset{!}{=} \mathtt{ok}.$$

Furthermore, for $\hat{K} \subseteq \Sigma^*$ and some bijective mapping to K, we define the asymmetric signature $\mathtt{sig} : X \times \hat{K} \to T$ and verification $\mathtt{test}^{\mathrm{a}} : X \times K \times S \to \{\mathtt{ok}, \mathtt{error}\}$, such that

$$\mathtt{test}^{\mathrm{a}}_k(x, \mathtt{sig}_{\hat{k}}(x)) \overset{!}{=} \mathtt{ok}.$$

Again, we assume the security property, that there is no easily computable direct mapping $X \to T$, nor $K \to \hat{K}$.

For signatures, we often write $\mathtt{sig}_k(t)$ instead of $\mathtt{sig}_{\hat{k}}(t)$ to highlight, that the public key k is known by all, even though the private key is needed to sign t.

Definition 1.1.3 (Cryptographic Primitives: Hashing)**.** For $X, T \subseteq \Sigma^*$, we define the bijective mapping $\mathtt{hash} : X \to T$, where $\mathtt{hash}^{-1}$ is not easily computable.

Definition 1.1.4 (Cryptographic Primitives: Nonces and Random Numbers)**.** Lastly, we assume, that we can generate perfectly random numbers or messages we call nonces, typically denoted by N.

1.2 Formal Protocol Model

Because we have difficulties when talking about security in an ambiguous model, we have to define a formal model to represent protocols, before defining what security even means.

In the context of this lecture, a formal model must support the following features: 1. The adversary must be able to control the routing of messages. 2. The model must be able to generate nonces. 3. The adversary may send arbitrary messages, limited only by cryptography. 4. The adversary must be able to control the scheduling.

Firstly, we have to define, how to notate messages and terms.

Definition 1.2.1 (Terms and Messages)**.** We denote the set of terms $\mathcal{T}$ as the smallest set with the following properties for $\mathcal{C}$ the set of constants, $\mathcal{V}$ the set of variables, and IDs the set of names.

$$
\begin{array}{rcccl}
 & & \{\epsilon\} \cup \mathcal{C} \cup \mathcal{V} \cup \mathrm{IDs} & \subseteq & \mathcal{T} \\
i \in \mathbb{N}, a \in \mathrm{IDs} & \Rightarrow & N_i, N_i^a, k_a, \hat{k}_a & \in & \mathcal{T} \\
t_1, t_2 \in \mathcal{T} & \Rightarrow & [t_1, t_2] & \in & \mathcal{T} \\
t, k \in \mathcal{T} & \Rightarrow & \mathtt{enc}^{\mathrm{s}}_k(t) & \in & \mathcal{T} \\
t \in \mathcal{T} \wedge a \in \mathrm{IDs} & \Rightarrow & \mathtt{enc}^{\mathrm{a}}_{k_a}(t) & \in & \mathcal{T} \\
t, k \in \mathcal{T} & \Rightarrow & \mathtt{mac}_k(t) & \in & \mathcal{T} \\
t \in \mathcal{T} \wedge a \in \mathrm{IDs} & \Rightarrow & \mathtt{sig}_{k_a}(t) & \in & \mathcal{T} \\
t \in \mathcal{T} & \Rightarrow & \mathtt{hash}(t) & \in & \mathcal{T}
\end{array}
$$

We relax this definition by allowing N_A or N''_A to be nonces generated by $A \in \mathrm{IDs}$ or writing $[t_0, [t_1, t_2]]$ as t_0, t_1, t_2. Terms can be represented as a tree or a directed acyclic graph (DAG).

We call terms without variables a *ground term* or a *message*.

In the tree or DAG of a term, we can trace to some position by *undoing* operators specified in Definition 1.2.1, we call these terms *subterms*.

Definition 1.2.2 (Subterms)**.** For some term $t \in \mathcal{T}$ and some position $p \in \mathbb{N}^+$, we write $t|_p$ as the subterm reached in t by following p.

More formally, we define the set of subterms $\mathrm{Sub}(t)$ of term $t \in \mathcal{T}$, as the smallest set with the following properties.

$$\begin{array}{rcrcl}
 & & t & \in & \mathrm{Sub}(t) \\
t' \in \mathrm{Sub}(t) & \Rightarrow & \mathrm{Sub}(t') & \subseteq & \mathrm{Sub}(t) \\
\exists t_1, t_2 : t = [t_1, t_2] & \Rightarrow & t_1, t_2 & \in & \mathrm{Sub}(t) \\
\exists k, t' : t = \mathtt{enc}^{\mathrm{s}}_k(t') & \Rightarrow & k, t' & \in & \mathrm{Sub}(t) \\
\exists k, t' : t = \mathtt{enc}^{\mathrm{a}}_k(t') & \Rightarrow & k, t' & \in & \mathrm{Sub}(t) \\
\exists k, t' : t = \mathtt{sig}_k(t') & \Rightarrow & k, t' & \in & \mathrm{Sub}(t) \\
\exists k, t' : t = \mathtt{mac}_k(t') & \Rightarrow & k, t' & \in & \mathrm{Sub}(t) \\
\exists t' : t = \mathtt{hash}(t') & \Rightarrow & t' & \in & \mathrm{Sub}(t)
\end{array}$$

We also define the set of subterms of a set of terms $S \subseteq \mathcal{T}$ as

$$\mathrm{Sub}(S) := \bigcup_{t \in S} \mathrm{Sub}(t).$$

Finally, we denote $\mathrm{Sub}^1(S)$ as the set of direct subterms of t.

As mentioned, the adversary may generate messages only limited by cryptography. Therefore they should not be able to send a message containing a non-derivable term. Intuitively, we define the *derivability* of some message as the *Dolev-Yao closure*.

Definition 1.2.3 (Dolev-Yao Closure)**.** For some $S \subseteq \mathcal{T}$, we define the Dolev-Yao closure of S, denoted $\mathrm{DY}(S)$ as the set of messages, that are derivable from S. The Dolev-Yao Closure of S is the smallest set with the following properties.

$$\begin{array}{rcr}
S \cup \{\epsilon\} \cup \mathrm{IDs} & \subseteq & \mathrm{DY}(S) \\
[t_1, t_2] \in \mathrm{DY}(S) & \Leftrightarrow & t_1, t_2 \in \mathrm{DY}(S) \\
a \in \mathrm{IDs}, t \in \mathrm{DY}(S) & \Rightarrow & \mathtt{enc}^{\mathrm{a}}_{k_a}(t) \in \mathrm{DY}(S) \\
t, k \in \mathrm{DY}(S) & \Rightarrow & \mathtt{enc}^{\mathrm{s}}_k(t), \mathtt{mac}_k(t) \in \mathrm{DY}(S) \\
t, \hat{k} \in \mathrm{DY}(S) & \Rightarrow & \mathtt{sig}_k(t) \in \mathrm{DY}(S) \\
\mathtt{enc}^{\mathrm{s}}_k(t), k \in \mathrm{DY}(S) & \Rightarrow & t \in \mathrm{DY}(S) \\
\mathtt{enc}^{\mathrm{a}}_{k_a}(t), \hat{k}_a \in \mathrm{DY}(S) & \Rightarrow & t \in \mathrm{DY}(S) \\
\mathtt{sig}_k(t) \in \mathrm{DY}(S) & \Rightarrow & t \in \mathrm{DY}(S) \\
\mathtt{mac}_k(t) \in \mathrm{DY}(S) & \Rightarrow & t \in \mathrm{DY}(S) \\
t \in \mathrm{DY}(S) & \Rightarrow & \mathtt{hash}(t)
\end{array}$$

It should be highlighted, that this definition of the Dolev-Yao closure allows composited keys, e.g., $\texttt{enc}^{\texttt{s}}_{N_A,N_B}(\texttt{"Alice loves Bob"})$ to communicate a secret between Alice and Bob. In this example, the message is derivable from $S \subseteq \mathcal{T}$, iff $N_A, N_B \in \mathrm{DY}(S)$, which is the case if both nonces are derivable or if $N_A, N_B \in S$.

Now, as seen pretty quickly, the Dolev-Yao closure is always infinite, therefore we can not compute it, but we can go around this problem by handling it as a decision problem.

Problem 1 (Derivation of a Term)**.** *The* DERIVE *problem has the input $S \subseteq \mathcal{T}, m \in \mathcal{T}$ and answers if $m \in DY(S)$ or not.*

It turns out, DERIVE is decidable in polynomial time using a fixed-point algorithm. The algorithm finds a shortest derivation of t iff $t \in \mathrm{DY}(S)$. This restricts the search space by using *composition* and *decomposition* rules which allow to derive to and from some term.

Definition 1.2.4 (Composition and Decomposition Rules)**.** For some term $m \in \mathcal{T}$, we define composition and decomposition rules as functions, that describe how m can be decomposed or composed—what to derive to with m and how to derive to m.

We denote decomposition and composition rules as $L_d(m), L_c(m) : R \to O$, where $R, O \subseteq \mathcal{T}$ are the required and obtained terms. This is also often written without set brackets, e.g., $L_c([a,b]) : a, b \to [a,b]$.

A whole derivation $D(m)$ of a message m from $S \subseteq \mathcal{T}$ is written as a chain of derivation rules, e.g.,

$$S = S_0 \to_{L_0} \to S_1 \to_{L_1} \cdots \to S_{n-1} \to_{L_{n-1}} \to S_n \ni m.$$

The success in this notation lies in the fact, that any required non-atomic terms are always direct subterms of the term the rule is applied on/to. More formally,

$$L_x(t) : R \to O \quad \Rightarrow \quad R \setminus \mathcal{T}_{\mathrm{at}} \subseteq \{t\} \cup \mathrm{Sub}^1(t)$$

Composition rules have exactly the terms of $\mathrm{Sub}^1(m)$ as required terms. Decomposition rules obtain only terms from $\mathrm{Sub}^1(m)$.

To reduce the search space even further, we can look at the properties of some shortest derivation $D_S(m)$ of $m \in \mathcal{T}$.

$$\begin{aligned} L_d(t) \in D_S(m) \quad &\Rightarrow \quad t \in \mathrm{Sub}(S) \\ L_c(t) \in D_S(m) \quad &\Rightarrow \quad t \in \mathrm{Sub}(S \cup \{m\}) \end{aligned}$$

To derive m from S, we only need decompositions of subterms from S or compositions of subterms of S or subterms of m.

From these rules, we find the following fixed-point algorithm to solve DERIVE (Problem 1) in polynomial time.

Algorithm 1 (Algorithm for DERIVE).

```
def derive(S, m):
    S_old = ∅
    while S_old ≠ S do
        S_old = S
        if ∃t ∈ Sub(S ∪ {m}) \ S, S → S ∪ {t} then
            S = S ∪ {t}
        fi
    od
    if m ∈ S then
        accept
    else
        reject
    fi
```

Definition 1.2.5 (Receive/Send Action)**.** We define a receive/send (r/s) action as some pair $(r, s) \in \mathcal{T}^2$ which we write $r \longrightarrow s$.

Currently, we still have some problems, because a protocol should be independent of its participants and needs features not yet implemented. For example, a protocol with some action $\mathtt{enc}^{\mathrm{a}}_{k_B}(A, x) \longrightarrow \mathtt{enc}^{\mathrm{a}}_{k_A}(x, N_B)$ assumes, that we have some way to handle variables, in this case x, and hard codes some recognition of key and nonce origin.

To fix these issues, we introduce *substitutions*.

Definition 1.2.6 (Substitutions)**.** Substitutions are functions $\sigma : \mathcal{V} \longrightarrow \mathcal{T}$ with $\exists^{<\infty} x : \sigma(x) \neq x$. A substitution σ is called *ground substitution*, if $\forall x \in \mathcal{V}, \sigma(x) \neq x : \sigma(x)$ is message.

We interpret a situation $\sigma(x) = x$ as the case, that x is uninitialized. Else, we evaluate inductively for arbitrary terms[1].

$$\begin{array}{lrcl} \text{IB: } x \in \mathcal{V} & \sigma(x) & & \text{is already defined} \\ \text{IS: } x \in \mathcal{T}_{\text{nat}} \setminus \mathcal{V} & \sigma(f(t_1, \ldots, t_n)) & := & f(\sigma(t_1), \ldots, \sigma(t_n)) \end{array}$$

Putting this together, we can model a protocol step, as a substitution state σ, next r/s action, incoming message m, updated substitution $\sigma \mapsto \sigma'$, and reply message m'.

Definition 1.2.7 (Matching)**.** We say a term $t \in \mathcal{T}$ matches with message m and substitution σ via substitution σ', if

$$\begin{aligned} \sigma'(t) &= m \\ \forall x, \sigma(x) \neq x : \sigma'(x) &= \sigma(x). \end{aligned}$$

[1] $\mathcal{T}_{\text{nat}}$ is supposed to be $\mathcal{T} \setminus \mathcal{T}_{\text{at}}$, aka., non-atomic terms.

1.2.1 Formal Protocol Model

Now we have all important steps to compose a formally sound protocol model.

Definition 1.2.8 (Protocol Instance)**.** A protocol instance is a $n \in \mathbb{N}$ long sequence $\mathcal{I} \in (\mathcal{T}^2)^n$ of r/s actions

$$\mathcal{I} = r_0 \rightarrow s_0, r_1 \rightarrow s_1, \ldots, r_{n-1} \rightarrow s_{n-1}$$

with $\forall i \in n : \mathcal{V}(s_i) \subseteq \bigcup_{j \leq i} \mathcal{V}(r_j)$, which basically means, that all variables of some response are already defined in previous received terms.

Definition 1.2.9 (Protocol)**.** A protocol consists of a finite number of instances $\mathcal{I}_0, \ldots, \mathcal{I}_{m-1}$ and a finite set $I \subseteq \mathcal{T}$ of messages, which models the initial adversary knowledge.

1.2.1.1 Needham-Schroeder Protocol

Using these definitions we can now finally define the Needham-Schroeder protocol using our formal notation.

$$\begin{array}{rcl}
\mathcal{I}_0 & & \text{Client (Alice)} \\
\epsilon & \rightarrow & \mathsf{enc}^{\mathrm{a}}_{k_B}(A, N_A) \\
\mathsf{enc}^{\mathrm{a}}_{k_A}(N_A, y) & \rightarrow & \mathsf{enc}^{\mathrm{a}}_{k_B}(y) \\
 & & \\
I_1 & & \text{Server (Bob)} \\
\mathsf{enc}^{\mathrm{a}}_{k_B}(A, x) & \rightarrow & \mathsf{enc}^{\mathrm{a}}_{k_A}(x, N_B) \\
 & & \\
I & & \text{Initial adversary knowledge} \\
I & = & \{A, B, C, k_A, k_B, k_C, \hat{k}_C, N_C^1, N_C^2, N_C^3\}
\end{array}$$

Now we need to model, how the adversary can attack a protocol. In this case, the protocol is insecure, because, if A starts the protocol with C, C can redirect the message to B, who thinks that A wants to talk with B and sends the response to A. Finally, A sends N_B to C, thinking that N_B was generated by C and C redirects N_B again to B. Now the secret $N_A \oplus N_B$ is known by A, B, and C even though B thinks, that only A also knows N_A and N_B. Thus, B may share secrets using the composited key $N_A \oplus N_B$ unbeknownst to C and not A.

1.2.2 Modeling Protocol Attacks

The current notation hard codes, that A and B want to talk with each other. Currently, the case that A wants to talk with C and B with A is not possible to model.

Additionally, the adversary must be able to control the scheduling, which is not yet implemented.

Definition 1.2.10 (Execution Order)**.** For protocol $P := (\{\mathcal{I}_0, \ldots, \mathcal{I}_{n-1}\}, I)$, we say each instance $I.$ has $|I.|$ steps. An *execution order* for P is a sequence[2] $o \in [n]^+$ such that each $j \in [n]$ appears at most $|I_j|$ times. We denotate $o(t)$ as the t-th element in o and $\#o(t) := |\{l \mid l < t \wedge o(l) = o(t)\}|$.

$o(t)$ can be seen as the current instance that gets *handled*, which makes o itself a sequence of instance steps. Therefore for some timestamp t, $\#o(t)$ is the current state of the referenced instance.

Definition 1.2.11 (Protocol Run / Attack)**.** For protocol $P := (\{\mathcal{I}_0, \ldots, \mathcal{I}_{n-1}\}, I)$ and all i-th rules of the j-th instance $\mathcal{I}_j$ being named[3] $r_i^j \longrightarrow s_i^j$, we define a *protocol run* of P or *attack* as an execution order o, and a substitution σ, such that

$$\forall t \in [|o|] : \sigma(r^{o(t)}_{\#o(t)}) \in \mathrm{DY}\left(I \cup \left\{\sigma(s^{o(l)}_{\#o(l)}) \mid l \in [t]\right\}\right).$$

In layman's terms, the substitution is defined such, that the values it can know from the received terms are derivable from previous send terms and the initial adversary knowledge. This is really practical, as the adversary knows every message that is send and received.

Now we still need to define, what a *successful* attack is, which is dependant on the definition of security, which is appropriate to the relevant context. As we discuss the security of protocols, we use the following definition.

Definition 1.2.12 (Successful Attack)**.** We call an attack on some protocol P successful, if

$$\texttt{FAIL} \in \mathrm{DY}\left(I \cup \left\{\sigma(s^{o(l)}_{\#o(l)}) \mid l \in [|o|]\right\}\right).$$

Now we can just introduce some r/s actions which receives a pair of the `BREAK` symbol with some hopefully underivable message, which sends `FAIL` on response.

We can thus extend the protocol definition in subsubsection 1.2.1.1 with the following r/s action for Bob, to model the security.

$$[\texttt{BREAK}, N_B] \quad \longrightarrow \quad \texttt{FAIL}$$

Currently, the model is still incomplete, as we still need to implement some things. For example, we want to keep partners flexible, e.g., allowing $A \longrightarrow C$ and $A \longrightarrow B$ in the same definition. We also want to allow multiple sessions of the protocol to be able to be simulated at the same time, which currently is not possible. We will fix both these problems on the way.

[2] In this lecture we define $[n]$ as $\{0, \ldots, n-1\}$

[3] We will reuse this notation.

1.3 Rusinowitch-Turuani Theorem

Our end goal is the automatic analysis of protocols. By this definition, this means solving Problem 2.

Problem 2 (Security of a Protocol)**.** *The* INSECURE *problem has input protocol P and decides if there is a successful attack on P or not.*

The way INSECURE is defined, we quickly run into undecidability for more complicated contexts, but the problem is in fact decidable and NP-complete for finite sessions. The proof of this, the Rusinowitch-Turuani Theorem, is the main goal of this section.

Theorem 1 (Rusinowitch-Turuani)**.** *INSECURE is NP-complete.*

1.3.1 Using Directed Acyclic Graphs to Model Terms

As some attacks may be exponential in length, we have to shorten the representation of terms and attacks to consider NP completeness. Therefore we redefine terms as DAGs.

Definition 1.3.1 (Terms as DAGs)**.** For some $S \subseteq \mathcal{T}$, we define the *DAG representation* of S as an edge-labeled graph $G := (V, E)$ with

$$\begin{aligned} V &:= \mathrm{Sub}(S) \\ E &:= \left\{ v_s \overset{\text{left}}{\to} v_e \in V^2 \mid \exists b \in \mathcal{T} : v_s \in \left\{ [v_e, b], \mathsf{enc}^{\mathrm{s}}_{v_e}(b), \mathsf{enc}^{\mathrm{a}}_{v_e}(b) \right\} \right\} \\ &\cup \left\{ v_s \overset{\text{right}}{\to} v_e \in V^2 \mid \exists b \in \mathcal{T} : v_s \in \left\{ [b, v_e], \mathsf{enc}^{\mathrm{s}}_{b}(v_e), \mathsf{enc}^{\mathrm{a}}_{b}(v_e) \right\} \right\} \end{aligned}$$

Moreover, we define $|S|_{\mathsf{DAG}} := |V|$.

The idea of this definition is to show, that if there exists some attack on a protocol P, then there exists an attack (o, σ), such that

$$\exists n \in \mathbb{N} : P \mapsto |\{\sigma(x) \mid x \in \mathcal{V}(P)\}|_{\mathsf{DAG}} \in \mathcal{O}(P \mapsto |P|^n).$$

We need this, in order to prove Theorem 1, as the solution substitution σ must not be larger than a polynomial dependant on $|P|$. Else, a solution can not be verified in polynomial time, making the problem too hard for NP-completeness.

Definition 1.3.2 (Notations for DAG Representations of Terms)**.** For $S \subseteq \mathcal{T}$ and substitution σ, we write $S\sigma := \{\sigma(t) \mid t \in S\}$. Additionally, for $t \in \mathcal{T}, x \in \mathcal{V}$, we write $[x \leftarrow t]$ as substitution with $x \mapsto t$ and $y \neq x \Rightarrow y \mapsto y$.

We need these notations, as thus we see, that for $S \subseteq \mathcal{T}$, $x \in \mathcal{V}$, t message, we have

$$|S[x \leftarrow t]|_{\mathsf{DAG}} \leq |S \cup \{t\}|_{\mathsf{DAG}}.$$

In other words, applying $x \leftarrow t$ on an arbitrary amount of variables in S is not more expensive than adding t directly to S.

A corollary of this is, that for $S \subseteq \mathcal{T}$ and σ a ground substitution on $x_1, \ldots, x_k \in \mathcal{V}$, we have

$$|S\sigma|_{\mathsf{DAG}} \leq |S \cup \{\sigma(x_1), \ldots, \sigma(x_k)\}|_{\mathsf{DAG}}.$$

Simply, we can substitute to compress the DAG of S further.

1.3.2 Short Attacks

As mentioned, we want to show, that for each successful attack, there is a short representation of it, for that we first have to define, what we even consider the size of an attack.

Definition 1.3.3 (Size of an Attack)**.** For an attack (o, σ) on a protocol P, we define the size $|\sigma|$ of the attack as $\sum_{x \in \mathcal{V}(P)} |\{\sigma(x)\}|_{\mathsf{DAG}}$.

A minimal attack is subsequently defined as an attack, with no smaller attack possible.

In most cases we have[4] $|\sigma(\cdot)|_{\mathsf{DAG}} = 1$, as often $\sigma(\cdot)$ is atomic and thus has no non-trivial subterms.

By the parsing lemma, if (o, σ) is a minimal successful attack on P, and there is some variable x with $|\sigma(x)|_{\mathsf{DAG}} > 1$, there is some term t with[5]

$$\begin{aligned} &t \in \mathrm{Sub}(\mathcal{T}(P)) \\ &t \notin \mathcal{V} \\ &\sigma(t) = \sigma(x) \end{aligned}$$

In other words, complex terms mapped from variables always match with terms in the protocol. Each term the attacker assigns to a variable is already present in the protocol.

The proof of the parsing lemma is available on `https://cloud.rz.uni-kiel.de/index.php/s/3osFeYcC6NwRMij?dir=/&editing=false&openfile=true`.

Finally, we can prove, that we can represent an attack (o, σ) on P in polynomial space, as we can show, that

$$\forall x \in \mathcal{V} : |\sigma(x)|_{\mathsf{DAG}} \leq |\mathcal{T}(P)|_{\mathsf{DAG}}.$$

Therefore, to prove Theorem 1, we only need to show, that INSECURE is NP-hard. Thus, we need to prove, that there is a polynomial reduction from another NP-hard problem to INSECURE.

[4] We write $|\sigma(\cdot)|_{\mathsf{DAG}}$ for $|\{\sigma(\cdot)\}|_{\mathsf{DAG}}$.

[5] $\mathcal{T}(P)$ is the set containing all terms of each r/s action in P.

The lecture shows this by reducing 3-SAT $\leq_p$ INSECURE. 3-SAT is defined[6] in Problem 3.

Problem 3 (3-SAT)**.** *The 3-SAT problem has input formula* $\phi := \bigwedge_{i \in [n]} (l_1^i \vee l_2^i \vee l_3^i)$ *for literals*[7] $l_j^i \in \{x_{r_{i,j}}, \overline{x}_{r_{i,j}}\}$ *for* $x_{r_{i,j}} \in \mathcal{V}$*. The problem decides if* ϕ *is satisfiable, i.e., there exists a substitution* $\sigma : \mathcal{V} \to \{\texttt{true}, \texttt{false}\}$*, such that* $\phi\sigma \to \texttt{true}$*.*

The reduction works by constructing a protocol P from formula ϕ in the following way. We create an instance A, which receives an assignment I from the adversary and distributes across all B_j instances using symmetric encryption. Each B_j now corresponds to $l_j^1 \vee l_j^2 \vee l_j^3$ and sends k_i to C, if the literal is satisfiable using I. There are 7 instances in each B_j, as for three literals, there are 7 possible instances, which satisfy the sentence. Finally, C sends `FAIL`, if it receives $k_1, \ldots, k_n$. The initial adversary knowledge is $\{\texttt{true}, \texttt{false}\}$.

Computing P from ϕ is obviously possible in polynomial time, and correctly accepts and rejects ϕ. Therefore, INSECURE is NP-hard and thus NP-complete.

1.4 Automatic Analysis

As mentioned, the goal of this chapter is the automatic analysis of security protocols. We quickly run into undecidability if we try to expand our formal model. E.g., allowing multiple and parallel sessions, makes INSECURE undecidable.

We also still have to handle, how the adversary controls the routing of messages and who chooses to initialize with whom.

We define a version of INSECURE, which allows parallel sessions in Problem 4.

Problem 4 (Security of an Unbound Protocol)**.** *The UNBOUNDED-INSECURE problem has input protocol P and decides, if there is an insecure protocol based*[8] *on P.*

Theorem 2 (Undecidability of Unrestricted Protocols)**.** *UNBOUNDED-INSECURE is undecidable.*

We show Theorem 2 by reducing PCP $\leq$ UNBOUNDED-INSECURE.

Problem 5 (Post Correspondence Problem)**.** *The PCP problem has input two lists of words* $\alpha, \beta \in (\{\texttt{0}, \texttt{1}\}^+)^n$*, and decides if there is a sequence* $I \in [n]^+$*, such that* $\bigtimes_{i \in I} \alpha_i = \bigtimes_{i \in I} \beta_i$*.*

Problem 5 is undecidable, by reduction from the halting problem. If there is no solution for given α and β, an algorithm will not halt.

PCP $\leq$ UNBOUNDED-INSECURE can be shown by constructing instances, which add words left or right to an intermediate solution using encryption. The attacker must find

[6] Hint: I redefined the problem in a way that feels more familiar to the notation used in the lecture.

[7] $r_{i,j}$ is the index of x that l_j^i is based on.

[8] In a way such that we only allow extensions of the protocol by copying instances and changing identities.

the correct execution order and right amount of instances, such that an entity receives encrypted $[x, x]$ and sends `FAIL`.

A satisfying result from Theorems 1 and 2 is, that while unbounded sessions are undecidable, a bounded number of sessions is decidable! Therefore, we can recursively enumerate all insecure protocols, as the problem is semi-decidable. An algorithm could e.g., search for an attack with sessions bounded to one, then two, three, etc. If the protocol is insecure, the algorithm *will* terminate, even though, this may take a long time. If the protocol is secure, the algorithm will not terminate.

This is still pretty unrealistic, as it is computationally expensive, as session count increases instance count and therefore input complexity. Therefore, we can use approximation algorithms. One idea is to over-estimate the attacker to reduce the problem space. Another idea is to model protocol properties in first-order Horn Logic. This is the underlying thought and the basis of *ProVerif*.

Definition 1.4.1 (Horn Logic)**.** We redefine the protocol notation in Horn Logic. The Dolev-Yao closure is defined as a one-dimensional relation d on $\mathcal{T}$. A term $t \in \mathcal{T}$ is defined to be derivable iff $d(t)$. Therefore we have facts such as $d(\hat{k}_C)$, as $\hat{k}_C \in I$.

All rules for the DY closure are written as first-order expressions[9], e.g.,

$$\forall t : d(t) \rightarrow d(\texttt{hash}(t)).$$

Security is equivalent to $\neg d(\texttt{FAIL})$.

The advantage of this notation is, that we can use logical rules between, e.g., literals $x_1, \ldots, x_n, y$, such as $(x_1 \wedge \cdots \wedge x_n \rightarrow y) \leftrightarrow (\neg x_1 \vee \cdots \vee x_n \vee y)$.

Definition 1.4.2 (Horn Translation of Protocol)**.** A protocol is written in Horn Logic by specifying rules for each instance in the original protocol.

For example, the Needham-Schoeder Protocol can be rewritten in the following way.

$$\begin{array}{lrcl}
A_1: & d(\epsilon) & \rightarrow & d(\texttt{enc}^{\mathrm{a}}_{k_C}(A, N_A)) \\
A_2: & \forall y : d(\texttt{enc}^{\mathrm{a}}_{k_A}(N_A, y)) & \rightarrow & d(\texttt{enc}^{\mathrm{a}}_{k_C}(y)) \\
B_1: & \forall x : d(\texttt{enc}^{\mathrm{a}}_{k_B}(A, x)) & \rightarrow & d(\texttt{enc}^{\mathrm{a}}_{k_A}(x, N_B)) \\
B_2: & d(\texttt{BREAK}, N_B) & \rightarrow & d(\texttt{FAIL})
\end{array}$$

Proving, that the Needham-Schoeder protocol is insecure can be done, be proving $d(\texttt{FAIL})$ by using the rules and assuming $d(\epsilon, k_A, k_B, k_C, \hat{k}_C, \texttt{BREAK}, A, B, C)$.

Definition 1.4.3 (Horn Clauses and Formulas)**.** We define Horn clauses to be disjunctions of literals, of which there is at most one positive literal. E.g., x_2 or $x_1 \vee \overline{x}_2 \vee \overline{x}_3$. The latter is equivalent ($\equiv$) to $x_2 \wedge x_3 \rightarrow x_1$. We also have $\overline{x}_1 \vee \overline{x}_2 \equiv \overline{x_1 \wedge x_2} \equiv x_1 \wedge x_2 \rightarrow \texttt{false}$.

A Horn formula is a conjunction of Horn clauses.

[9] The notation changes to be more familiar to traditional logic.

The idea is basically to write derivation steps and rules as Horn clauses and evaluate a horn formula. For this, the lecture additionally defines unifiers, resolvents, and resolutions, and states defines an algorithm to test a horn formula on satisfiability. Because this is not mentioned any further, and there are no proofs on the aforementioned theorem, this content is not included in this summary.

1.4.1 ProVerif

ProVerif is a security analysis tool which can be used to prove different securities of protocols, even with unbounded protocol sessions, albeit non-deterministic. The language has a strong type system.

The following are some basic keywords and their semantics.

```
# Free typed algebraic variables
free var
# Communication channel Type
ch: channel
# Data Type
data: bitstring
# Variable the adversary can not directly access.
private
# Defines the security property.
query
```

A basic insecure handshake protocol can be specified in the following way. Some types used need to be defined with, e.g., `type skey` for the private key. The functions for encryption and signing are also defined manually by introducing reduction rules.

```
# Channel for A and B.
free c:channel

# The attacker should not be able to derive FAIL.
free FAIL:bitstring [private]
query attacker (FAIL)

# Alice (Client)
let clientA(pkA: pkey, skA:skey, pkB:spkey) =
    # Send identifier and public key.
    out (c, pkA)
    # When some term is received,
    in (c, x:bitstring)
    # and we decrypt it with A's key,
    let y = adec(x, skA) in
        # and we can verify, that we received k_B,k by B,
        let (=pkB, k:key) = checksign(y, pkB) in
            # send FAIL.
            out (c, senc(FAIL, k))

```

```
# Bob (Server)
let serverB(pkB: spkey, skB: sskey) =
    # When we receive some identifier and a public key,
    in (c, pkX:pkey)
    # generate a new key,
    new k:key
    # and send own public and new key, signed and encrypted for recipient.
    out (c, aenc(sign((pkB, k), skB), pkX))
    # If we receive some string we can not decrypt with k, ignore.
    in (c, x:bitstring)
    let z = sdec(x, k) in 0

# Main process
# Generate private and signing keys.
new skA: skey
new skB: sskey
# Distribute derived public keys.
let pkA = pk(skA) in out(c, pkA)
let pkB = spk(skB) in out(c, pkB)

# Run Alice's and Bob's procedures an unbounded number of sessions.
( (!clientA(pkA, skA, pkB)) | (!serverB(pkB, skB)) )
```

As mentioned, we have to manually define, how the symmetric and asymmetric encryptions work. This is done in ProVerif by defining so-called *equational theories*.

Definition 1.4.4 (Equational Theory)**.** In ProVerif we can define primitives, e.g, the symmetric encryption, very similar to how we defined them in Definition 1.1.1.

$$\forall t \in T : \mathsf{dec}^{\mathsf{s}}_k(\mathsf{enc}^{\mathsf{s}}_k(x)) = x$$

This property is called an equational theory, a set of equational theories is often denoted as E.

More formally, we define the relation $\twoheadrightarrow_E$ between two terms, if the right term can be derived from the left term using a rule in E. The relation $\twoheadrightarrow^*_E$ is the closure of $\twoheadrightarrow_E$ under transitivity, reflexivity, and application of function symbols. The relation $\equiv_E$ is defined as the symmetric closure of $\twoheadrightarrow^*_E$.

We define some derivation $\twoheadrightarrow_E$ *terminating*, if there is no infinite sequence using rules in E and *confluent*, if there are multiple orders in which rules are used to derive some term. A set of rules E is defined *convergent*, if $\twoheadrightarrow_E$ is terminating and confluent. E is a *convergent subterm theory*, if E is convergent and for all derivations, the right-hand side is a subterm of the left-hand side.

Finally, we say a term $t \in \mathcal{T}$ is in *E-normal form*, if $\forall t \twoheadrightarrow_E t' : t = t'$, i.e., we can not simplify t further.

Theorem 3 (Algorithms for Convergent Equational Theories)**.**

1. *If a set E of rules is convergent, then for each $t \in \mathcal{T}$ there exists a $[t] \in \mathcal{T}$, such that $[t]$ is in normal form and $t \equiv_E [t]$. We call $[t]$ the normal form of t.*

2. $\forall t, t' \in \mathcal{T} : t \equiv_E t' \Leftrightarrow [t] = [t']$.

3. *We can decide if* $t \equiv_E t'$ *and build the DAG representation of* $[t]$ *in polynomial time.*

Using this notation, we can define asymmetric encryption in ProVerif.

```
type skey
type pkey
fun pk(skey): pkey
fun aenc(bitstring, pkey): bitstring
reduc forall m: bitstring, k: skey; adec(aenc(m,pk(k)),k) = m
```

Realistically, for some two terms, that the adversary can not simplify, we would want them to look the same, i.e., random bits. We model this in the following way.

Definition 1.4.5 (Tests for Indistinguishability)**.** For adversary knowledge $S \subseteq \mathcal{T}$, we define an S-*test* as a pair $(M, M') \in \mathcal{T}^2$, such that $M, M' \in \mathrm{DY}(S)$, and $\exists^1 x \in \mathcal{V} : x \in M, M'$.

A message t *satisfies* test (M, M'), if $M[t/x] \equiv_E M'[t/x]$. We call two messages S-*indistinguishable* if there is no S-test satisfied by exactly one of them.

By this definition, the adversary can distinguish $\mathtt{enc}^{\mathrm{a}}_k(\mathtt{true})$ and $\mathtt{enc}^{\mathrm{a}}_k(\mathtt{false})$. We can fix this, by introducing random nonces into our encryption primitives which are not known by the adversary. The decision problem that covers, if two messages are S-distinguishable, is decidable in polynomial time.

This notion enhances our security definition based on INSECURE whether we can derive some secret to the question, if a secret can be guessed by distinguishability. We call this *strong secrecy*, which is also covered by ProVerif.

A different security measure regarding if our protocol is still secure in the future, when some private key will be calculated from the public key by the adversary, is known as *forward secrecy*. It can also be modeled in ProVerif.

Yet another security definition regarding the order a protocol is being executed is *correspondence*. As an example consider a key exchange protocol, where a key is generated by Alice and accepted by Bob. For this example, we can define a security based on the fact, that the key is secure exactly when Alice accepts the key implies that it was generated by Bob. The order of these events, i.e., $\mathtt{accept}(A, B, k)$ and $\mathtt{generate}(A, B, k)$, matter!

Chapter 2

Information Flow

This is the second part of the lecture, where we will define and discuss different security properties on networks.

One example where security was violated by an information flow mess up was the *Meltdown* attack, where CPU pipelining lead to a process gaining access to forbidden memory. In this example, the security policy is isolation between (unrelated) processes. Processes may communicate with the kernel, but not directly with each other, this was violated by the attack.

2.1 Formal Model

To formalize the idea, that information may only flow between allowed actors, we will have to go through some definitions.

Definition 2.1.1 (Security Policy)**.** For a set of security domains (agents) D, a *security policy* is a reflexive and often transitive relation $\rightarrowtail \subseteq D^2$. For domains $L, H \in D$, we write $L \rightarrowtail H$, that information may flow from L to H. Subsequently, $L \not\rightarrowtail H$ describes, that information must not flow from L to H.

A standard policy is $\rightarrowtail = \{(L, H)\}$, written $L \rightarrowtail H$, where we only allow information to flow from L to H but not the other way around. This is named the H/L policy.

Definition 2.1.2 (System)**.** We model a system used by agents as a state machine automata. Transitions model different actions agents can take.

A system is a tuple $(S, s_0, A, \texttt{step}, D, O, \texttt{obs}, \texttt{dom})$, where

$$\begin{aligned} S &\mathrel{\hat{=}} \text{Set of states} \\ s_0 \in S &\mathrel{\hat{=}} \text{Initial state} \\ A &\mathrel{\hat{=}} \text{Set of actions} \\ \texttt{step} : S \times A \to S &\mathrel{\hat{=}} \text{Step function} \end{aligned}$$

$$
\begin{aligned}
D &\mathrel{\hat{=}} \text{Set of agents/security domains} \\
O &\mathrel{\hat{=}} \text{Set of possible observations} \\
\texttt{obs} : S \times D \to O &\mathrel{\hat{=}} \text{Observation function} \\
\texttt{dom} : A \to D &\mathrel{\hat{=}} \text{Domain function}
\end{aligned}
$$

For actor L and state s, we define $\texttt{obs}_L(s)$ as the observation of L in s, i.e., the information L can derive from s.

For state $s \in S$ and sequence of actions $\alpha \in A^*$, we define $s \cdot \alpha$ as the state obtained by performing all actions in α.

2.2 Security Models

There are different notions of how to define security in the information flow context, the lecture mainly discusses *P-security* and *IP-security*.

Definition 2.2.1 (Purge Function)**.** For some set of domains $E \subseteq D$, a sequence of actions $\alpha \in A^*$, and policy $\rightarrowtail$, we define $\alpha \upharpoonleft E$ as the subsequence of actions $\beta \in A^*$ from α, where

$$\forall i \in [|\beta|] : \texttt{dom}(\beta_i) \in E.$$

In other words, $\alpha \upharpoonleft E$ is α without actions not owned by some actor in E.

Moreover we define the *purge function* $\texttt{purge} : A^* \times D \to A^*$ with

$$\texttt{purge} : \alpha, u \mapsto \alpha \upharpoonleft \{v \in D \mid v \rightarrowtail u\}.$$

We write $\texttt{purge}_u(\alpha)$. This is the subsequence of α owned by some domain from which information may flow to u. $\texttt{purge}_u(\alpha)$ contains all actions in α, which u may directly know of. On important thing to remember in this notion is, that most of the times specify that $\rightarrowtail$ is reflexive and transitive.

Now we can finally define P-security.

Definition 2.2.2 (P-Security)**.** A system $(S, s_0, A, \texttt{step}, D, O, \texttt{obs}, \texttt{dom})$ is P-secure with respect to a policy $\rightarrowtail$, iff

$$\forall u \in D, s \in S, \alpha_1, \alpha_2 \in A^* : \texttt{purge}_u(\alpha_1) = \texttt{purge}_u(\alpha_2) \Rightarrow \texttt{obs}_u(s \cdot \alpha_1) = \texttt{obs}_u(s \cdot \alpha_2)$$

Informally, this means that u should always know exactly the same for two different sequences of actions with the same $\texttt{purge}_u$.

P-security of the system is equivalent with

$$\forall s \in S, u \in D, \alpha \in A^* : \texttt{obs}_u(s \cdot \alpha) = \texttt{obs}_u(S \cdot \texttt{purge}_u(\alpha)).$$

Well now we have a different problem. When we want to verify, that a system is P-secure (or later, IP- or TA-secure), we may have a finite amount of state pairs $|S^2| = |S|^2$, but sequences can be arbitrary long. Therefore, we must find a way to find suitable sequence pairs until no change occurs anymore. To fix this issue, we define another way to find P-security and prove that this definition is equivalent, and that we can use this definition to verify P-security in polynomial time.

Definition 2.2.3 (Unwinding)**.** An *unwinding* of a system $(S, s_0, A, \texttt{step}, D, O, \texttt{obs}, \texttt{dom})$ with policy $\rightarrowtail$ is a collection of relations $(\sim_u)_{u \in D}$ with

$$\begin{aligned} s \sim_u t &\Rightarrow \texttt{obs}_u(s) = \texttt{obs}_u(t) && \text{(OC)} \\ s \sim_u t &\Rightarrow s \cdot a \sim_u t \cdot a && \text{(SC)} \\ \texttt{dom}(a) \not\rightarrowtail u &\Rightarrow s \sim_u s \cdot a && \text{(LR)} \end{aligned}$$

These properties are called *output consistency*, *step consistency*, and *left respect*.

To construct an unwinding on a system, we can just add the trivial reflexive cases, and then close the relation under transitivity, symmetry, left respect, and step consistency until nothing changes. Finally, to verify that we have indeed found an unwinding, we check for output consistency.

Now we look at IP-security, a security measure, where we don't require transitivity. Instead, we accept intransitive noninterference.

Definition 2.2.4 (Intransitive Policies)**.** For IP-security, we may specify *intransitive policies*, such as $H \rightarrowtail T \rightarrowtail L$, for $H, T, L \in D$, where we allow T to transmit some information from H to L, but L may not find out about H's actions directly.

For this we have to define a new purge function—the *immediate purge* function.

Definition 2.2.5 (Immediate Purge Function)**.** To define the purge function used in IP-security, we firstly have to define another function, which is used by the former. We define the `sources` function as the set of agents which may interfere with some user in some sequence. It is similar to a cumulative domain, but a bit more complex. For all $u \in D, a \in A, \alpha \in A^*$, we define the $\texttt{sources} : A^* \times D \rightarrow \mathcal{P}(D)$ function with

$$\begin{aligned} \epsilon, u &\mapsto \{u\} \\ a\alpha, u &\mapsto \begin{cases} \texttt{sources}(\alpha, u) \cup \{\texttt{dom}(a)\} & \exists v \in \texttt{sources}(\alpha, u) : \texttt{dom}(a) \rightarrowtail v, \\ \texttt{sources}(\alpha, u) & \text{else.} \end{cases} \end{aligned}$$

In other words, this is a subset of the cumulative domain over α, where we require, that there are policies which allow u to know information of the domain. The important thing to note here, is that we read α from right to left when evaluating $\texttt{sources}(\alpha, u)$, e.g., for

policy $H \rightarrowtail T \rightarrowtail L$ we have $\texttt{sources}(thl, L) = \{T, L\}$, because $\texttt{sources}(hl, L) = \{L\}$ and $H \not\rightarrowtail L$ as the policy is intransitive. The set contains only the users whose actions may be forwarded to u, when the entire trace is performed. Instead of $\texttt{sources}(\cdot, u)$ we also write $\texttt{sources}_u(\cdot)$.

Now we can finally define the immediate purge function $\texttt{ipurge} : A^* \times D \to A^*$. It is defined inductively for all $u \in D, a \in A, \alpha \in A^*$ with

$$\begin{aligned} \epsilon, u &\mapsto \epsilon \\ a\alpha, u &\mapsto \begin{cases} a\texttt{ipurge}(\alpha, u) & \texttt{dom}(a) \in \texttt{sources}(a\alpha, u), \\ \texttt{ipurge}(\alpha, u) & \text{else.} \end{cases} \end{aligned}$$

In other words, the immediate purge function looks at the current trace of α, i.e. its current source, and keeps the action a iff it is owned by an actor u currently knows about.

We also allow $\texttt{ipurge}(\cdot, u)$ to be written as $\texttt{ipurge}_u(\cdot)$.

Definition 2.2.6 (IP-Security)**.** Similarly to before, we define a system IP-secure with respect to intransitive policy $\rightarrowtail$ iff

$$\forall u \in D, s \in S : \texttt{ipurge}_u(\alpha_1) = \texttt{ipurge}_u(\alpha_2) \Rightarrow \texttt{obs}_u(s \cdot \alpha_1) = \texttt{obs}_u(s \cdot \alpha_2).$$

This is pretty similar to P-security—this scheme is even used for TA-security, we just replace the relevant purge function.

As the immediate purge function is *stricter* than the normal purge function, we see that P-security implies IP-security. This also goes for TA-security, which this lecture does not formally define.

We can again define an unwinding for IP-security and decide if there is one in polynomial time, proving or refuting IP-security.

Definition 2.2.7 (IP-Unwinding)**.** An *IP-unwinding* of a system $(S, s_0, A, \texttt{step}, D, O, \texttt{obs}, \texttt{dom})$ with intransitive policy $\rightarrowtail$ is a family of equivalence relations $(\sim_u^v)_{u,v \in D}$ on S, such that

$$\begin{aligned} s \sim_u^v t &\Rightarrow \texttt{obs}_u(s) = \texttt{obs}_u(t) && \text{(IP-OC)} \\ s \sim_u^v t \wedge v \not\rightarrowtail \texttt{dom}(a) &\Rightarrow s \cdot a \sim_u^v t \cdot a && \text{(IP-SC)} \\ v \not\rightarrowtail u \wedge \texttt{dom}(a) = v &\Rightarrow s \sim_u s \cdot a && \text{(IP-LR)} \end{aligned}$$

In this family, some v is known to perform to u potentially secret operations. u is a low-level observer.